

REACT NATIVE CON EXPO



Profesora: Yanina A. Dacal Villamayor

Styled Components

¿QUÉ ES?

CSS-in-JS para React Native

- Estilos encapsulados en componentes
- Reutilizables en toda la aplicación
- Soporte completo para theming
- Sintaxis familiar de CSS

Instalación:

```
npm install styled-components
```

Styled Components - Ejemplo Básico (Parte 1)

IMPORTACIÓN Y CONTAINER

```
import styled from 'styled-components/native';  
const Container = styled.View flex: 1;  
padding: 20px;;
```

Explicación:

- Container es un View estilizado
- flex: 1 hace que ocupe todo el espacio disponible
- padding: 20px añade espacio interno en todos los lados
- Los estilos se escriben como template literals

Styled Components - Ejemplo Básico (Parte 2)

COMPONENTE TITLE CON ESTILOS

```
const Title = styled.Text font-size: 24px;  
font-weight: bold;;
```

Explicación:

- Title es un componente Text estilizado
- font-size: 24px define el tamaño del texto
- font-weight: bold aplica negrita al texto
- Ideal para títulos y encabezados destacados
- Reutilizable en toda la aplicación

Styled Components - Tarjeta de Usuario (Parte 1)

CÓDIGO DEL CARD Y NAME

```
const Card = styled.View background-color: white;
```

```
padding: 16px;
```

```
border-radius: 10px;
```

```
elevation: 3;;
```

```
const Name = styled.Text font-size: 18px;
```

```
font-weight: bold;;
```

→ Card: contenedor con fondo blanco y sombra

→ Name: texto destacado para el nombre

Styled Components - Tarjeta de Usuario (Parte 2)

EMAIL Y COMPONENTE USERCARD

```
const Email = styled.Text color: gray;;  
export default function UserCard({ user }) {  
  return (  
  
    {user.name}  
    {user.email}  
  
  );  
}
```

- Email es un componente Text con color gris
- UserCard recibe un objeto user como prop
- Muestra nombre y email usando los estilos definidos
- Componente reutilizable para mostrar datos de usuario

ThemeProvider

¿QUÉ ES?

ThemeProvider es un componente de styled-components que permite compartir estilos globales en toda tu aplicación.

- Compartir estilos globales entre componentes
- Cambiar temas dinámicamente en tiempo real
- Ideal para implementar modo claro/oscuro
- Centraliza colores, fuentes y espaciados
- Facilita el mantenimiento de estilos consistentes

ThemeProvider - Definir Temas

DEFINICIÓN DE TEMAS CLARO Y OSCURO

```
export const lightTheme = {  
  colors: {  
    background: '#fff',  
    text: '#000'  
  }  
};  
  
export const darkTheme = {  
  colors: {  
    background: '#000',  
    text: '#fff'  
  }  
};
```


ThemeProvider - Uso Dinámico (Parte 1)

ALTERNAR TEMAS CON ESTADO

```
import { ThemeProvider } from 'styled-components/native';  
const App = () => {  
  const [isDark, setIsDark] = useState(false);  
  const theme = isDark ? darkTheme : lightTheme;  
  return (  
  
    <MainApp toggleTheme={() => setIsDark(!isDark)} />  
  
  );  
};
```

Explicación: El estado isDark controla qué tema está activo.
ThemeProvider envuelve la app y pasa el tema a todos los

ThemeProvider - Uso Dinámico (Parte 2)

CÓDIGO PARA USAR TEMA EN COMPONENTE STYLED

```
const Container = styled.View background-color: ${({ theme }) =>
theme.colors.background};
const TextStyled = styled.Text color: ${({ theme }) =>
theme.colors.text};;
```

Explicación: Los colores se obtienen dinámicamente del tema activo.
Cuando el tema cambia, los componentes se actualizan automáticamente.

Context API

¿PARA QUÉ SIRVE?

Context API permite compartir estado global en tu aplicación sin necesidad de pasar props manualmente en cada nivel.

- Compartir estado global

Datos accesibles desde cualquier componente

- Evitar "prop drilling"

No más cadenas interminables de props

- Ideal para datos como:
 - Usuario autenticado
 - Configuración de la app
 - Tema (claro/oscuro)

Context API - Crear Context

CREAR Y PROVEER USERCONTEXT

```
import { createContext, useState } from 'react';
const UserContext = createContext();
export function UserProvider({ children }) {
  const [user, setUser] = useState(null);
  const login = (name) => setUser({ name });
  const logout = () => setUser(null);
  return (
    <UserContext.Provider value={{ user, login, logout }}>
      {children}
    </UserContext.Provider>
  );
}
```

Context API - Usar en Componentes

ACCESO AL CONTEXTO DESDE CUALQUIER COMPONENTE

```
import { useContext } from 'react';  
function LoginScreen() {  
  const { login } = useContext(UserContext);  
  return (  
    <Button  
      title="Entrar"  
      onPress={() => login('Yanina')}  
    />  
  );  
}
```

- useContext accede al contexto sin prop drilling
- login está disponible en cualquier componente hijo

AsyncStorage - ¿Qué es?

PERSISTENCIA DE DATOS LOCAL

- Guardar datos en el dispositivo
- Persistir información entre sesiones
- Ideal para recordar sesión del usuario
- Almacenamiento clave-valor asíncrono

Instalación:

```
expo install @react-native-async-storage/async-storage
```


AsyncStorage - Guardar y Leer

FUNCIONES PARA PERSISTIR DATOS

```
import AsyncStorage from '@react-native-async-storage/async-storage';
const saveUser = async (user) => {
  try {
    await AsyncStorage.setItem('user', JSON.stringify(user));
  } catch (e) {
    console.log('Error:', e);
  }
};
const getUser = async () => {
  try {
    const data = await AsyncStorage.getItem('user');
    return data ? JSON.parse(data) : null;
  } catch (e) {
    return null;
  }
};
```

AsyncStorage - Eliminar y Uso en Context

PERSISTENCIA AUTOMÁTICA DE SESIÓN

Código para eliminar datos y cómo integrar en Context API:

```
const removeUser = async () => {  
  await AsyncStorage.removeItem('user');  
};  
  
export function UserProvider({ children }) {  
  const [user, setUser] = useState(null);  
  useEffect(() => {  
    getUser().then(savedUser => {  
      if (savedUser) setUser(savedUser);  
    });  
  }, []);  
  
  const login = async (name) => {  
    const newUser = { name };  
    setUser(newUser);  
    await saveUser(newUser);  
  };  
}
```

```
const logout = async () => {  
  setUser(null);  
  await removeUser();  
};  
  
return (  
  <UserContext.Provider value={{ user, login, logout }}>  
    {children}  
  </UserContext.Provider>  
);  
}
```

Explicación: removeUser elimina datos guardados.
UserProvider ahora persiste la

SQLite - ¿Qué es?

BASE DE DATOS LOCAL PARA REACT NATIVE

- Base de datos local completa en el dispositivo
- Ideal para datos estructurados y operaciones CRUD
- Persistencia robusta para apps offline
- Consultas SQL estándar

Instalación:

```
expo install expo-sqlite
```

SQLite - Crear Tabla

ABRIR BASE DE DATOS Y CREAR ESTRUCTURA

```
import * as SQLite from 'expo-sqlite';  
const db = SQLite.openDatabase('myapp.db');  
const initDB = () => {  
  db.transaction(tx => {  
    tx.executeSql(  
      CREATE TABLE IF NOT EXISTS users (  
        id INTEGER PRIMARY KEY AUTOINCREMENT,  
        name TEXT,  
        email TEXT  
      );  
    );  
  });  
};
```

- openDatabase: abre o crea la base de datos
- CREATE TABLE IF NOT EXISTS: crea tabla solo si no existe

SQLite - Insertar Datos

AGREGAR USUARIOS A LA BASE DE DATOS

```
const insertUser = (name, email) => {  
  db.transaction(tx => {  
    tx.executeSql(  
      'INSERT INTO users (name, email) VALUES (?, ?)',  
      [name, email],  
      (_, result) => {  
        console.log('Usuario agregado:', result.insertId);  
      }  
    );  
  });  
};
```

Explicación: La función insertUser recibe nombre y email como parámetros. Usa transaction para ejecutar el INSERT con placeholders (?) para evitar inyección SQL. El callback

SQLite - Consultar Datos

SELECT * FROM USERS

```
const getUsers = (callback) => {  
  db.transaction(tx => {  
    tx.executeSql(  
      'SELECT * FROM users',  
      [],  
      (_, { rows: { _array } }) => {  
        callback(_array);  
      }  
    );  
  });  
};
```

Explicación: Esta función ejecuta una consulta SQL

SQLite - Actualizar y Eliminar

UPDATE Y DELETE POR ID

```
const updateUser = (id, name) => {  
  db.transaction(tx => {  
    tx.executeSql(  
      'UPDATE users SET name = ? WHERE id = ?',  
      [name, id]  
    );  
  });  
};
```

```
const deleteUser = (id) => {  
  db.transaction(tx => {  
    tx.executeSql(  
      'DELETE FROM users WHERE id = ?',  
      [id]  
    );  
  });  
};
```

- UPDATE modifica el nombre del usuario por ID
- DELETE elimina el registro completo por ID

TouchableOpacity

INTRODUCCIÓN

¿Qué es TouchableOpacity?

- Componente para crear botones con feedback visual
- Reduce la opacidad del elemento al presionar
- Proporciona retroalimentación táctil intuitiva
- Muy utilizado en apps reales de React Native
- Alternativa más flexible que el componente Button
- Permite personalizar completamente el contenido interno



TouchableOpacity - Botón Reutilizable

COMPONENTE BUTTON CONFIGURABLE

```
const Button = ({ title, onPress, color = '#ff4081' }) => (  
  <TouchableOpacity  
    onPress={onPress}  
    style={{  
      backgroundColor: color,  
      padding: 12,  
      borderRadius: 8,  
      alignItems: 'center',  
      margin: 10  
    }}  
  )
```

```
    activeOpacity={0.7}  
  <Text style={{ color: 'white', fontWeight: 'bold' }}>  
    {title}  
  </Text>
```

- title: texto del botón
- onPress: función al presionar
- color: color de fondo personalizable
- activeOpacity: nivel de transparencia al tocar

Imágenes - Local y Remota

CARGANDO IMÁGENES EN REACT NATIVE

Imagen Local con require():

```
<Image  
  source={require('./assets/logo.png')}  
  style={{ width: 100, height: 100 }}  
  resizeMode="contain"  
>
```

Imagen Remota con uri:

```
<Image  
  source={{ uri: 'https://example.com/photo.jpg' }}  
  style={{ width: 200, height: 200 }}  
  resizeMode="cover"  
>
```

- contain: escala manteniendo proporción
- cover: llena el espacio recortando si es necesario



Imágenes - Avatar Circular

STYLED.IMAGE CON BORDER-RADIUS

```
import styled from 'styled-components/native';  
const Avatar = styled.Image width: 80px;  
  height: 80px;  
  border-radius: 40px;  
  border-width: 2px;  
  border-color: #007AFF;;  
// Uso  
<Avatar source={{ uri: user.image }} />
```

Explicación: Estilo típico para fotos de perfil con borde circular y color destacado.

ScrollView

CONTENEDORES SCROLLEABLES

ScrollView permite crear contenido desplazable cuando excede el tamaño de la pantalla.

// Scroll vertical

```
<ScrollView
```

```
  showsVerticalScrollIndicator={false}
```

```
  contentContainerStyle={{ padding: 20 }}
```

Contenido largo...

// Scroll horizontal

```
<ScrollView
```

```
  horizontal
```

```
  showsHorizontalScrollIndicator={false}
```

```
<View style={{ width: 100 }} />
```

```
<View style={{ width: 100 }} />
```

Desplazamiento horizontal y vertical en ScrollView

TextInput - Básico

INPUT CONTROLADO CON ESTADO

```
import { TextInput } from 'react-native';
import { useState } from 'react';
const [text, setText] = useState("");
<TextInput
  placeholder="Ingresa tu nombre"
  value={text}
  onChangeText={setText}
  style={{
    borderWidth: 1,
    padding: 12,
    borderRadius: 8
  }}
/>
```

- useState controla el valor del input
- value muestra el estado actual
- onChangeText actualiza el estado

TextInput - Tipos Especiales

EMAIL, CONTRASEÑA Y TELÉFONO

// Email

// Contraseña

// Teléfono

Explicación: props específicas para distintos tipos de entrada de datos del usuario.

Formulario Completo - Parte 1

ESTADOS Y FUNCIÓN SUBMIT

```
import { ScrollView, TextInput, Button } from 'react-native';
import { useState } from 'react';
export default function FormScreen() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const handleSubmit = () => {
    alert(Bienvenido ${name}!);
  };
}
```

Explicación: Se definen tres estados para los campos del formulario y una función handleSubmit que muestra una alerta de bienvenida.

Formulario Completo - Parte 2

RENDERIZADO DEL FORMULARIO COMPLETO

```
return (  
<ScrollView style={{ flex: 1, padding: 20 }}>  
  <TextInput  
    placeholder="Nombre completo"  
    value={name}  
    onChangeText={setName}  
    style={{ borderWidth: 1, padding: 12,  
      borderRadius: 8, marginBottom: 10 }}  
  />  
  
  );
```

```
<TextInput  
  placeholder="Email"  
  value={email}  
  onChangeText={setEmail}  
  keyboardType="email-address"  
  style={{ borderWidth: 1, padding: 12,  
    borderRadius: 8, marginBottom: 10 }}  
/>  
  
<TextInput  
  placeholder="Contraseña"  
  value={password}  
  onChangeText={setPassword}  
  secureTextEntry  
  style={{ borderWidth: 1, padding: 12,  
    borderRadius: 8, marginBottom: 10 }}  
/>
```

Organización - Estructura Principal

ÁRBOL DE CARPETAS DEL PROYECTO

```
/my-app
├── /src
│   ├── /components
│   ├── /screens
│   ├── /navigation
│   ├── /context
│   ├── /services
│   ├── /utils
│   ├── /theme
│   └── /assets
├── App.js
└── package.json
```

Estructura escalable y clara para proyectos React Native con Expo.

Organización - Components

ESTRUCTURA MODULAR DE COMPONENTES

/components

├── /Button

| ├── Button.js

| ├── styles.js

| └── index.js

├── /Card

| ├── Card.js

| ├── styles.js

| └── index.js

├── /Avatar

| ├── Avatar.js

| ├── styles.js

| └── index.js

Cada componente tiene su carpeta con:

- Archivo principal (.js) - lógica del componente
- styles.js - estilos con styled-components

¡GRACIAS!

React Native con Expo - Guía Práctica Completa
¡Ahora estás listo para crear apps móviles increíbles!